

Heimdall

Optimizing Storage I/O Admission with Extensive Machine
Learning Pipeline

EuroSys '25 Presentation

Daniar H. Kurniawan, Rani Ayu Putri, Peiran Qin, Kahfi S. Zulkifli, Ray A. O. Sinurat, Janki Bhimani, Sandeep

Madireddy, Achmad Imam Kistijantoro, Haryadi S. Gunawi

Presented by Kailer Laino

Background: The Storage Challenge

Flash Storage & SSDs

Solid State Drives (SSDs) are the backbone of modern data centers. While they offer high performance, their internal operations are complex and non-deterministic.

Tail Latency & GC

Internal processes like Garbage Collection (GC) and wear leveling cause significant "tail latency" spikes. A fast request can suddenly become very slow if it hits a busy drive.

Motivation



Negative Impact

I/O misses and tail latencies have large negative effects on overall system performance and user experience.



Failing Heuristics

Current State-of-the-Art (SotA) models and heuristics are not performing as well as promised on modern, complex workloads.

Accuracy Gap

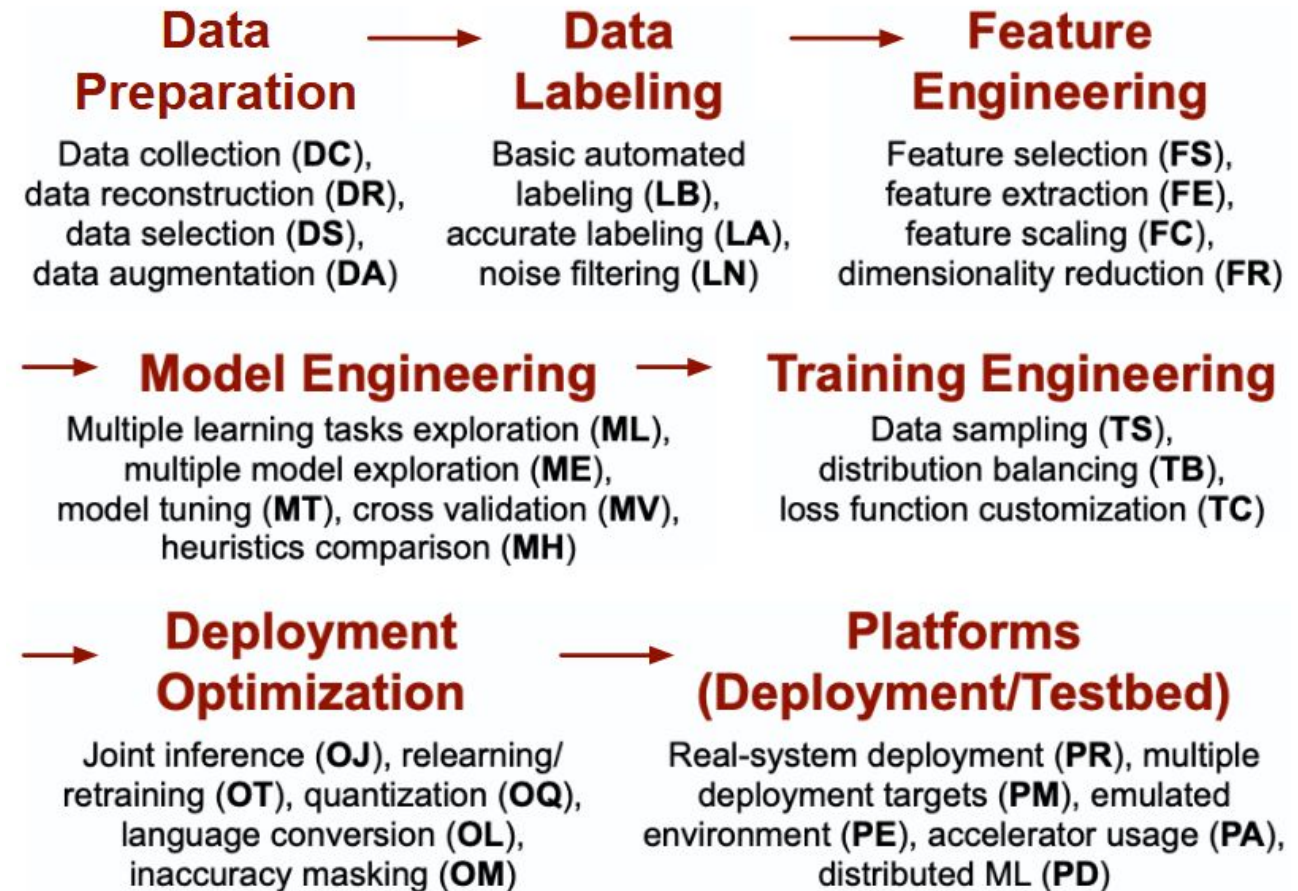
There is a critical need for a more accurate method to predict "slow" vs "fast" I/O periods to route requests effectively.

The Heimdall Approach

Extensive ML Pipeline

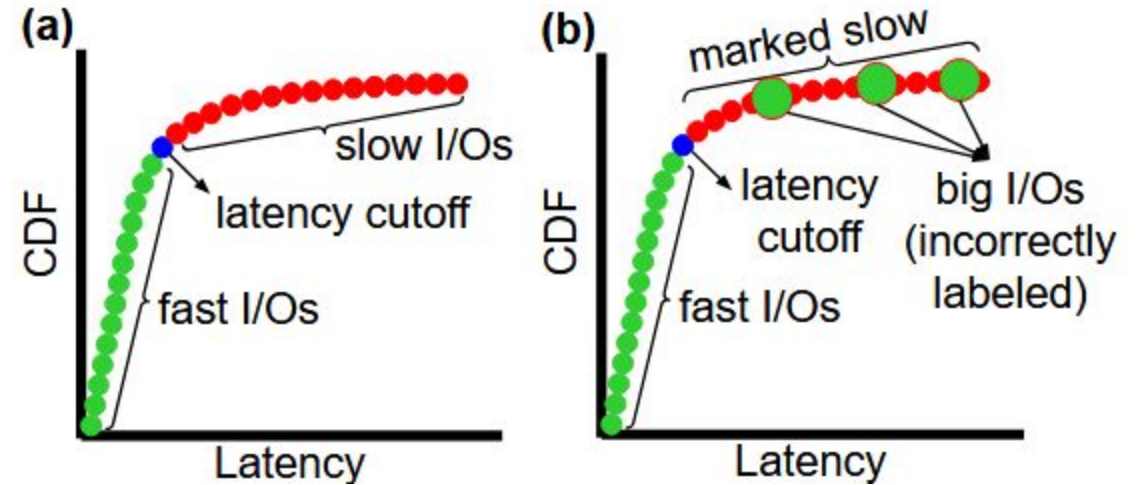
Heimdall moves beyond simple models by implementing a comprehensive pipeline.

- **Data Preparation:** Cleaning raw traces.
- **Labeling:** Period-based accurate labeling.
- **Feature Engineering:** Extracting critical signals.
- **Model Engineering:** Neural Network optimization
- **Deployment:** Joint inference for speed.

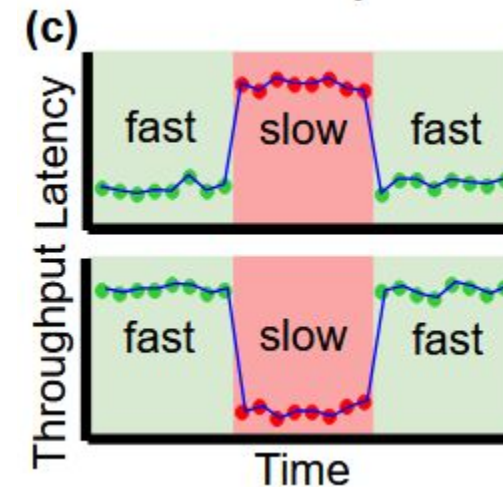


Period-Based Labeling

Traditional methods use a simple **latency cutoff** to label data as "fast" or "slow." This is often inaccurate for variable-sized I/Os.

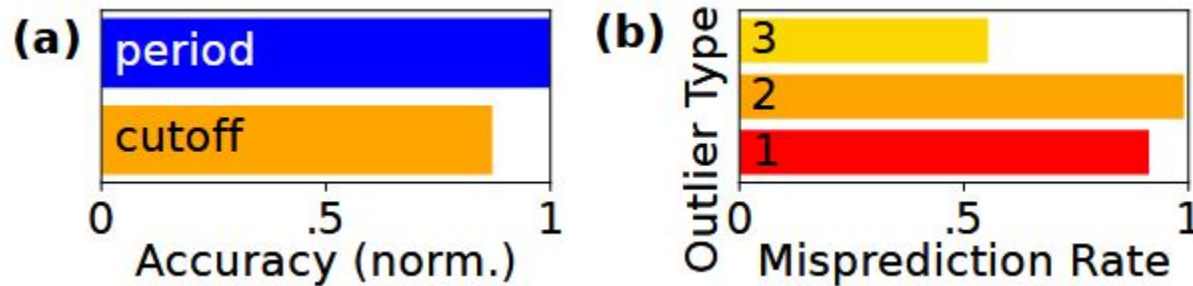


Uses **Period-based labeling**. It identifies "slow periods" where the device is internally busy (e.g., during GC) and "fast periods" where it is free. This creates a much more accurate ground truth for training. (results shown later)



Contribution: 3-Stage Noise Filtering

- ▼ **Outliers within Slow Period:** Filters out "lucky" fast requests that happen during a busy period (e.g., cache hits), which would confuse the model.
- ⚡ **Outliers within Fast Period:** Filters out random slow requests during a fast period that are not due to contention (e.g., transient errors).
- ✂ **Short Noises:** Removes very short bursts of noise that do not represent true device contention. This clean data significantly improves accuracy.



Neural Network & Features

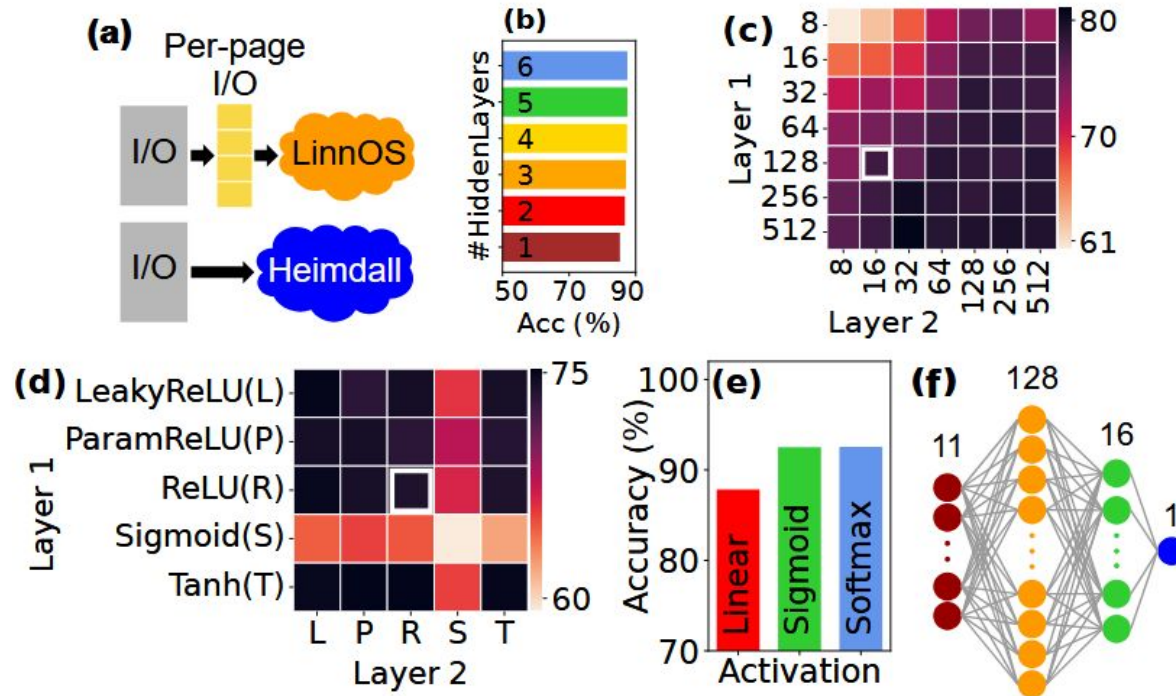


Figure 9. Hyperparameter tuning (§3.5). (a) Per-page vs. per-I/O, (b) hidden layers, (c) number of layers, (d) activation functions, (e) output layer, and (f) final NN design.

Optimized Architecture

Heimdall uses a lightweight **Neural Network** (NN) with 2 hidden layers. Feature engineering identified the most critical inputs, allowing for a model that offers the best balance of accuracy and stability.

Deployment Optimization

Inference Optimization

Manual conversion from Python-to-C++ ($20\mu\text{s}$)

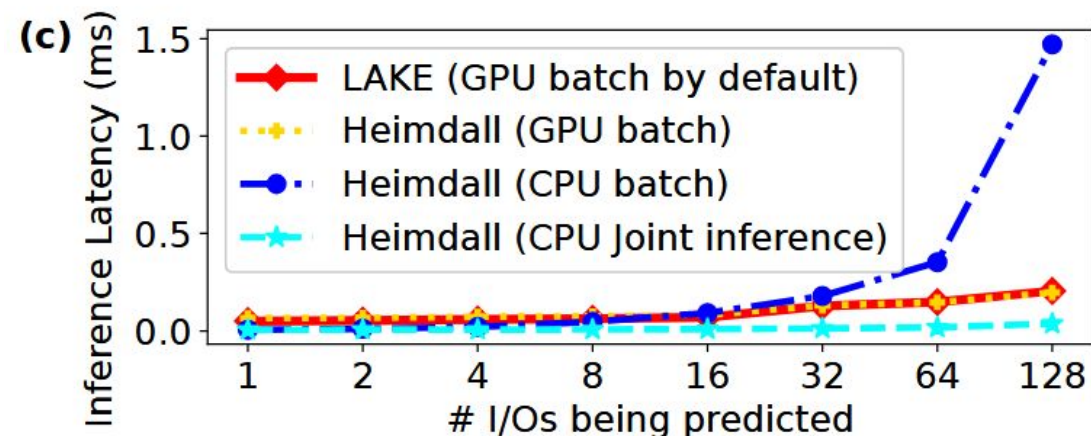
gcc optimization, -O3 ($0.08\mu\text{s}$)

Quantization ($0.05\mu\text{s}$)

Joint Inference

To reduce CPU overhead, Heimdall employs **Joint Inference**. Instead of predicting for every single I/O, it can predict for a group of requests at once.

Claims this works better than batching.



Key Results

93%

Decision Accuracy

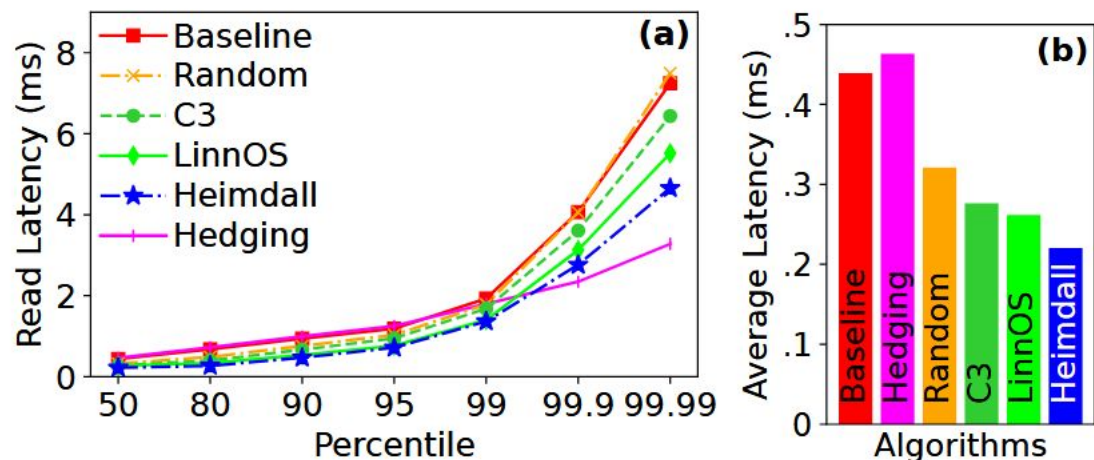
35%

Lower Avg Latency

28KB

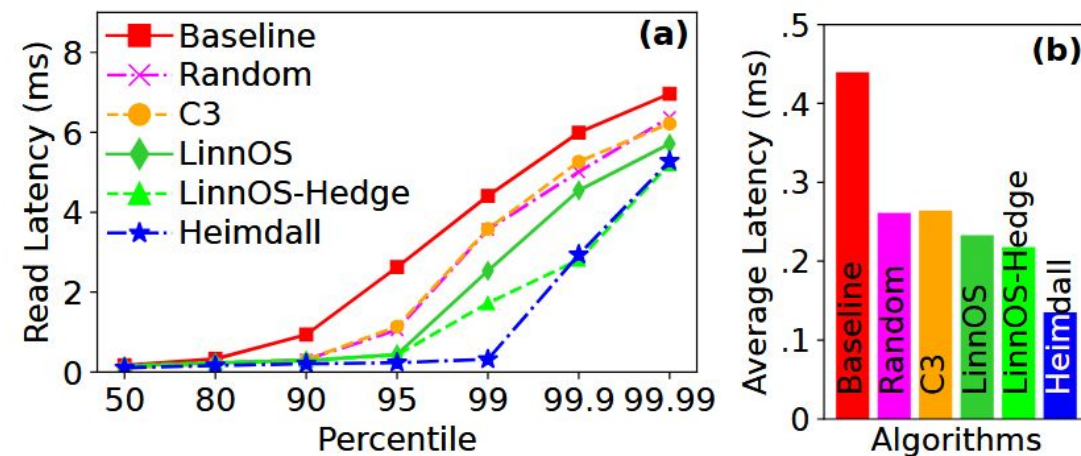
Memory
Overhead

Performance Comparison (on 3 different levels)



User-Level I/O (sim)

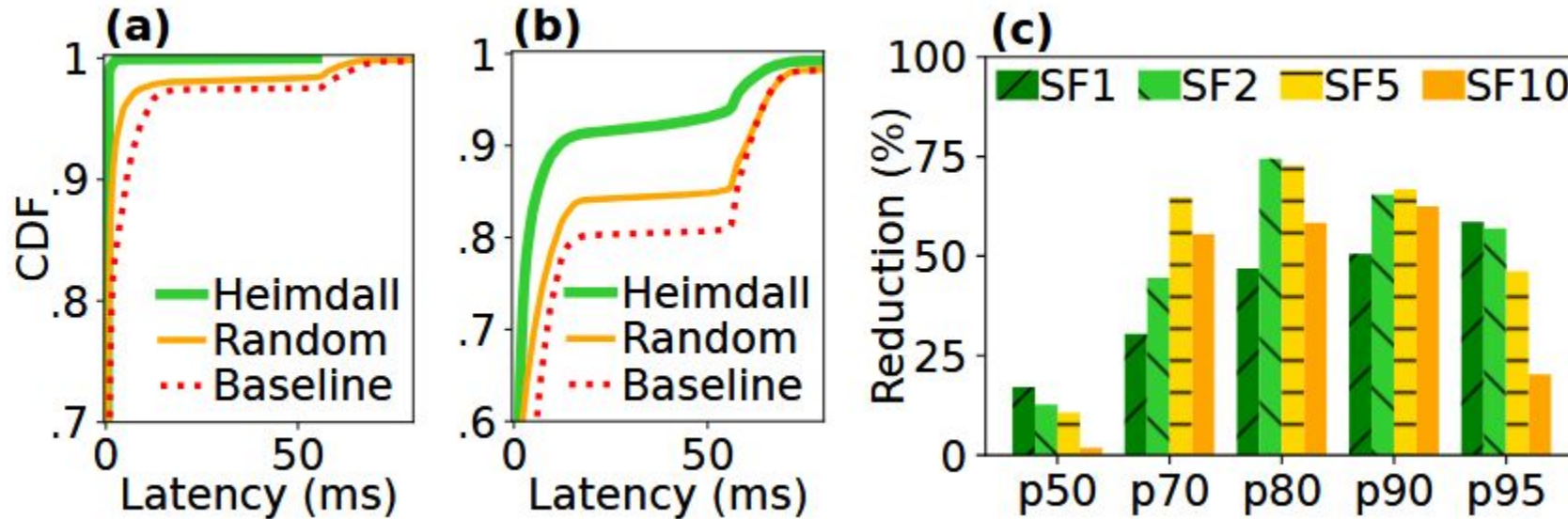
Evaluation across 500 traces shows Heimdall consistently beats both heuristics (C3) and previous ML (LinnOS). Hedging beats out Heimdall above p99, but its average latency is much worse than Heimdall.



Kernel-Level I/O

Deployed in the Linux block layer, Heimdall achieves **38–48% lower average latency** on heterogeneous consumer-grade SSDs.

Performance Comparison cont.



As the **Scaling Factor** increases (more sub-requests), tail latency usually spikes early. Heimdall efficiently cuts this tail, maintaining low latency where random load-balancing and baselines fail.

In distributed systems like Ceph, a user request is split into multiple sub-requests. If even one sub-request hits a slow node, the entire user request is delayed.

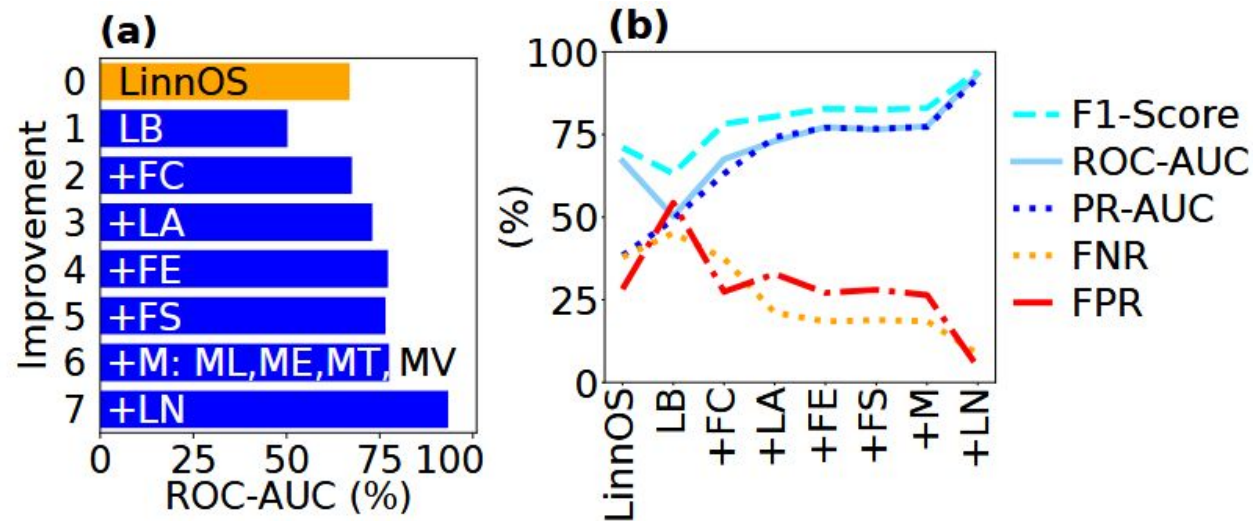
Accuracy

Breakdown

Accurate Labeling (+5.5%): Switching from simple latency cutoffs to Period-based labeling provided the initial boost.

Feature Engineering (+4.0%): Adding critical features like I/O Size and Throughput refined the model.

Noise Filtering (+16.0%): The largest gain came from removing "lucky" fast I/Os (outliers) that were confusing the model during training.



Critique

Strengths

Comprehensive: Covers the entire ML pipelines, not just the model

High Performance: Significant gains in accuracy and latency reduction.

Practical: Successfully deployed in Linux Kernel and Ceph.

Weaknesses

Complexity: The multi-stage pipeline is complex to implement and requires immense domain knowledge

Drift: Performance can drift over time, requiring retraining (although this is addressed in Sec 7)

Data Dependency: Heavily relies on high-quality trace data.

Future Work



AutoML

Developing an automatic
Heimdall pipeline to reduce
manual tuning effort.



Retraining

Implementing continual learning
strategies to handle workload
drift over long deployments.



Expansion

Applying the pipeline approach
to other storage problems like
caching and prefetching.

Questions?